EA2. Creación de repositorio: procesamiento funcional de datos para el sistema de transporte urbano TecnoMovil Data

Jhoan Esteban Pérez Henao

Institución Universitaria Digital de Antioquia

Programación Orientada a Objetos II

Docente: Leonardo Montes Gutiérrez

Noviembre de 2026

Introducción

En esta actividad se trabajó el caso de estudio TecnoMovil Data. En la ciudad ficticia de TecnoValle el sistema de transporte público genera millones de registros al día, cada vez que una persona pasa su tarjeta por un validador al entrar o salir de una estación. El sistema que tenían estaba hecho con programación imperativa: muchos ciclos, variables que se iban modificando y listas compartidas, lo que causaba errores y demoras al procesar la información.

Lo que se pidió fue rediseñar ese procesamiento usando programación funcional en Java, sin frameworks. Para eso se usaron funciones puras, objetos inmutables, expresiones lambda, funciones de orden superior y la API de Streams. Con esas herramientas se resolvieron los seis requerimientos del caso: afluencia por estación, horas pico, rutas más utilizadas, patrones de viaje por usuario, tiempo promedio entre estaciones y detección de sobrecarga en rutas. Finalmente, el proyecto se subió a un repositorio de GitHub en la rama principal (main).

En este informe explico cómo quedó organizado el código, qué hace cada parte y por qué cumple con los principios de la programación funcional, junto con las capturas de la ejecución y el enlace al repositorio.

Objetivos

Objetivo general

Aplicar los conceptos de programación funcional en Java para procesar los datos del sistema de transporte TecnoMovil Data de forma declarativa, y alojar la solución en un repositorio de GitHub en la rama principal.

Objetivos específicos

- Usar funciones puras, inmutabilidad, lambdas y funciones de orden superior en un caso realista.

- Resolver los seis requerimientos del caso de estudio con la API de Streams (filter, map, collect, groupingBy, counting).
- Entender cómo manejar muchos datos de manera declarativa, diciendo qué se quiere obtener y no paso a paso cómo recorrerlos.
- Crear un repositorio en GitHub y subir la solución, el informe y el ZIP del proyecto en la rama main.

Desarrollo

Estructura del proyecto

El proyecto se hizo en Java 21 sin librerías externas. Quedó con tres clases dentro del paquete tecnomovil:

- **RegistroTransporte:** es un record con los datos de cada registro: idUsuario, ruta, estacion, accion (entrada o salida) y timestamp.
- **ServicioTransporte:** tiene las funciones puras que resuelven los requerimientos a) al f).
- **Main:** crea 20 registros de prueba que simulan un día de operación y muestra los resultados en consola.

Conceptos de programación funcional aplicados

Inmutabilidad. Un objeto inmutable es uno que no se puede cambiar después de creado. Por eso RegistroTransporte es un `record`: todos sus atributos son final y no tiene setters. Además, los datos de prueba se crean con `List.of(...)` y los resultados filtrados con `.toList()`, que devuelven listas inmutables. Al final del programa imprimo el tamaño de la lista original para comprobar que sigue con los mismos 20 registros después de todo el procesamiento.

Funciones puras. Una función pura siempre devuelve el mismo resultado si recibe los mismos datos y no produce efectos secundarios (no modifica variables de afuera, no imprime, no guarda nada). Todos los métodos de ServicioTransporte son así: reciben la lista, la procesan con un stream y retornan un resultado nuevo. Los `System.out.println` solo están en Main.

Lambdas. Son funciones cortas sin nombre que se escriben directamente donde se necesitan. Por ejemplo, `r -> r.accion().equals("entrada")` indica qué registros se quieren conservar, y `r -> r.timestamp().getHour()` indica que se agrupe por la hora. También usé referencias a métodos como `RegistroTransporte::estacion`, que es una forma más corta de escribir una lambda.

Funciones de orden superior. Son funciones que reciben otra función como parámetro. Hice dos: `filtrar(lista, Predicate)`, donde quien la llama decide el criterio con una lambda, y `contarPor(lista, Function)`, que agrupa y cuenta por el campo que se le indique. Esta última la reutilizo en los requerimientos a), c) y f), así no repito el mismo código tres veces.

Streams. Permiten procesar la colección como una cadena de operaciones sin escribir ciclos for. Usé `filter` para filtrar, `map` para transformar, `sorted` para ordenar, `collect` con `groupingBy` y `counting` para agrupar y contar, y `average` para promediar. Como las funciones son puras, el mismo procesamiento se puede hacer con `parallelStream()` sin riesgo; en Main lo comprobé contando las entradas de las dos formas y da el mismo resultado.

Solución de cada requerimiento

a) Afluencia por estación. Primero filtro solo los registros de tipo "entrada" con la función filtrar y una lambda, y luego cuento cuántos hay por estación con contarPor.

b) Horas pico. Agrupo los registros por la hora del timestamp y cuento cuántos hay en cada hora. Después busco el valor máximo y devuelvo la hora (o las horas, si hay empate) que tienen ese máximo.

c) Rutas más utilizadas. Cuento los registros por ruta y los ordeno de mayor a menor con sorted y comparingByValue().reversed().

d) Patrones de viaje por usuario. Ordeno todos los registros por hora y los agrupo por usuario, guardando solo el nombre de la estación con mapping. Así queda la lista de estaciones de cada usuario en orden cronológico.

e) Tiempo promedio entre estaciones. Para cada usuario tomo cada "entrada" seguida de su "salida" y calculo los minutos entre las dos con Duration.between. Usé IntStream.range para recorrer las posiciones sin un for, y al final saco el promedio con average.

f) Sobrecarga en rutas. Cuento los registros por ruta y, si una ruta supera el umbral, la marco como "CRITICA"; si no, como "NORMAL". El umbral se recibe por parámetro (en la prueba usé 8).

Código fuente

A continuación está el código completo de las tres clases, igual al que está en el repositorio.

RegistroTransporte.java

```
package tecnomovil;

import java.time.LocalDateTime;

/**
 * Representa un registro del sistema de transporte de TecnoValle.
 * Cada vez que un usuario pasa la tarjeta por un validador se genera uno.
 *
 * Use un record porque sus atributos son final: una vez se crea el objeto
 * no se puede modificar (inmutabilidad). Asi ninguna funcion puede
 * cambiar los datos originales por error.
 */
public record RegistroTransporte(
        String idUsuario,
```

```
        String ruta,
        String estacion,
        String accion,          // "entrada" o "salida"
        LocalDateTime timestamp
) {
}
```

ServicioTransporte.java

```
package tecnomovil;

import java.time.Duration;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.function.Function;
import java.util.function.Predicate;
import java.util.stream.Collectors;
import java.util.stream.IntStream;

/**
 * Aqui estan todas las funciones que procesan los registros.
 *
 * Todas son funciones puras:
 *  - Solo usan lo que reciben por parametro.
 *  - No modifican la lista original (los streams crean resultados nuevos).
 *  - No imprimen ni guardan nada, solo retornan un resultado.
 * Por eso, si les paso los mismos datos, siempre devuelven lo mismo.
 */
public class ServicioTransporte {

    // ---------------- Funciones de orden superior ----------------

    /**
     * Funcion de orden superior: recibe otra funcion (un Predicate) como parametro.
     * El criterio de filtro lo decide quien la llama, usando una lambda.
     */
    public static List<RegistroTransporte> filtrar(List<RegistroTransporte> registros,
                                                   Predicate<RegistroTransporte> criterio) {
        return registros.stream()
                .filter(criterio)
                .toList(); // toList() devuelve una lista inmutable
    }

    /**
     * Funcion de orden superior: recibe una Function que dice por que campo agrupar
     * (estacion, ruta, usuario...) y cuenta cuantos registros hay en cada grupo.
     * La reutilizo en varios requerimientos para no repetir codigo.
     */
    public static Map<String, Long> contarPor(List<RegistroTransporte> registros,
                                              Function<RegistroTransporte, String> campo) {
        return registros.stream()
                .collect(Collectors.groupingBy(campo, TreeMap::new, Collectors.counting()));
    }

    // ---------------- a) Afluencia por estacion ----------------

    /** Cuenta cuantas personas ENTRAN a cada estacion. */
    public static Map<String, Long> afluenciaPorEstacion(List<RegistroTransporte> registros) {
        List<RegistroTransporte> entradas = filtrar(registros, r ->
r.accion().equals("entrada"));
        return contarPor(entradas, RegistroTransporte::estacion);
```

```
    }

    // ---------------- b) Horas pico ----------------

    /** Agrupa los registros por la hora del dia (0 a 23) y cuenta cuantos hay en cada hora. */
    public static Map<Integer, Long> registrosPorHora(List<RegistroTransporte> registros) {
        return registros.stream()
                .collect(Collectors.groupingBy(r -> r.timestamp().getHour(),
                        TreeMap::new, Collectors.counting()));
    }

    /** Devuelve las horas pico: las horas que tienen la mayor cantidad de registros. */
    public static List<Integer> horasPico(List<RegistroTransporte> registros) {
        Map<Integer, Long> porHora = registrosPorHora(registros);
        long maximo = porHora.values().stream()
                .mapToLong(Long::longValue)
                .max()
                .orElse(0);
        return porHora.entrySet().stream()
                .filter(e -> e.getValue() == maximo)
                .map(Map.Entry::getKey)
                .toList();
    }

    // ---------------- c) Rutas mas utilizadas ----------------

    /** Devuelve las rutas ordenadas de la mas usada a la menos usada. */
    public static List<Map.Entry<String, Long>> rutasMasUtilizadas(List<RegistroTransporte>
registros) {
        return contarPor(registros, RegistroTransporte::ruta).entrySet().stream()
                .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
                .toList();
    }

    // ---------------- d) Patrones de viaje por usuario ----------------

    /** Para cada usuario, lista las estaciones por donde paso, en orden de hora. */
    public static Map<String, List<String>> patronesDeViaje(List<RegistroTransporte> registros)
{
        return registros.stream()
                .sorted(Comparator.comparing(RegistroTransporte::timestamp))
                .collect(Collectors.groupingBy(RegistroTransporte::idUsuario, TreeMap::new,
                        Collectors.mapping(RegistroTransporte::estacion,
Collectors.toList())));
    }

    // ---------------- e) Tiempo promedio entre estaciones ----------------

    /**
     * Calcula el tiempo promedio (en minutos) entre una entrada y la salida
     * siguiente del mismo usuario, es decir, lo que dura el viaje entre dos estaciones.
     */
    public static double tiempoPromedioEntreEstaciones(List<RegistroTransporte> registros) {
        Map<String, List<RegistroTransporte>> porUsuario = registros.stream()
                .sorted(Comparator.comparing(RegistroTransporte::timestamp))
                .collect(Collectors.groupingBy(RegistroTransporte::idUsuario));

        return porUsuario.values().stream()
                .flatMap(viajes -> IntStream.range(0, viajes.size() - 1)
                        .filter(i -> viajes.get(i).accion().equals("entrada")
                                && viajes.get(i + 1).accion().equals("salida"))
                        .mapToObj(i -> Duration.between(viajes.get(i).timestamp(),
                                viajes.get(i + 1).timestamp()).toMinutes()))
                .mapToLong(Long::longValue)
                .average()
```

```
                .orElse(0);
    }

    // ---------------- f) Sobrecarga en rutas ----------------

    /** Marca cada ruta como "CRITICA" si sus registros superan el umbral, o "NORMAL" si no. */
    public static Map<String, String> estadoDeRutas(List<RegistroTransporte> registros, long
umbral) {
        return contarPor(registros, RegistroTransporte::ruta).entrySet().stream()
                .collect(Collectors.toMap(
                        Map.Entry::getKey,
                        e -> e.getValue() > umbral ? "CRITICA" : "NORMAL",
                        (a, b) -> a,
                        TreeMap::new));
    }
}
```

Main.java

```
package tecnomovil;

import java.time.LocalDateTime;
import java.util.List;

/**
 * Clase principal. Crea unos datos de prueba que simulan un dia del sistema
 * de transporte de TecnoValle y muestra el resultado de cada requerimiento.
 * Aqui si se imprime en consola; las funciones de ServicioTransporte no imprimen nada.
 */
public class Main {

    public static void main(String[] args) {

        // List.of crea una lista inmutable: nadie le puede agregar ni quitar elementos.
        List<RegistroTransporte> registros = List.of(
                reg("U01", "R1", "Central",     "entrada", 6, 50),
                reg("U01", "R1", "Universidad", "salida",  7, 20),
                reg("U02", "R2", "Poblado",     "entrada", 7, 0),
                reg("U02", "R2", "Industriales","salida",  7, 25),
                reg("U03", "R1", "Central",     "entrada", 7, 10),
                reg("U03", "R1", "Caribe",      "salida",  7, 40),
                reg("U04", "R3", "San Antonio", "entrada", 7, 30),
                reg("U04", "R3", "Cisneros",    "salida",  7, 45),
                reg("U05", "R1", "Central",     "entrada", 7, 45),
                reg("U05", "R1", "Universidad", "salida",  8, 15),
                reg("U01", "R1", "Universidad", "entrada", 12, 10),
                reg("U01", "R1", "Central",     "salida",  12, 40),
                reg("U06", "R2", "Envigado",    "entrada", 17, 15),
                reg("U06", "R2", "Poblado",     "salida",  17, 35),
                reg("U02", "R2", "Industriales","entrada", 17, 45),
                reg("U02", "R2", "Poblado",     "salida",  18, 10),
                reg("U07", "R1", "Caribe",      "entrada", 17, 50),
                reg("U07", "R1", "Central",     "salida",  18, 20),
                reg("U03", "R1", "Caribe",      "entrada", 18, 0),
                reg("U03", "R1", "Central",     "salida",  18, 30)
        );

        System.out.println("=== TecnoMovil Data - Procesamiento funcional ===");
        System.out.println("Registros cargados: " + registros.size());

        // a)
        System.out.println("\na) Afluencia por estacion (entradas):");
        ServicioTransporte.afluenciaPorEstacion(registros)
```

```
                .forEach((estacion, total) -> System.out.println("   " + estacion + " -> " +
total));

        // b)
        System.out.println("\nb) Registros por hora:");
        ServicioTransporte.registrosPorHora(registros)
                .forEach((hora, total) -> System.out.println("   " + hora + ":00 -> " +
total));
        System.out.println("   Hora(s) pico: " + ServicioTransporte.horasPico(registros));

        // c)
        System.out.println("\nc) Rutas mas utilizadas:");
        ServicioTransporte.rutasMasUtilizadas(registros)
                .forEach(e -> System.out.println("   " + e.getKey() + " -> " + e.getValue() + "
registros"));

        // d)
        System.out.println("\nd) Patrones de viaje por usuario:");
        ServicioTransporte.patronesDeViaje(registros)
                .forEach((usuario, estaciones) -> System.out.println("   " + usuario + ": " +
estaciones));

        // e)
        System.out.printf("%ne) Tiempo promedio entre estaciones: %.1f minutos%n",
                ServicioTransporte.tiempoPromedioEntreEstaciones(registros));

        // f)
        long umbral = 8;
        System.out.println("\nf) Estado de las rutas (umbral: mas de " + umbral + "
registros):");
        ServicioTransporte.estadoDeRutas(registros, umbral)
                .forEach((ruta, estado) -> System.out.println("   " + ruta + " -> " + estado));

        // Orden superior con una lambda definida aqui mismo
        List<RegistroTransporte> manana = ServicioTransporte.filtrar(registros,
                r -> r.timestamp().getHour() < 9);
        System.out.println("\nExtra) Registros antes de las 9:00 (filtro con lambda): " +
manana.size());

        // parallelStream: como las funciones son puras, el resultado es el mismo en paralelo
        long entradasSecuencial = registros.stream().filter(r ->
r.accion().equals("entrada")).count();
        long entradasParalelo = registros.parallelStream().filter(r ->
r.accion().equals("entrada")).count();
        System.out.println("Extra) Entradas con stream: " + entradasSecuencial
                + " | con parallelStream: " + entradasParalelo);

        // Inmutabilidad: la lista original no cambio despues de todo el procesamiento
        System.out.println("Extra) Registros en la lista original al final: " +
registros.size());
    }

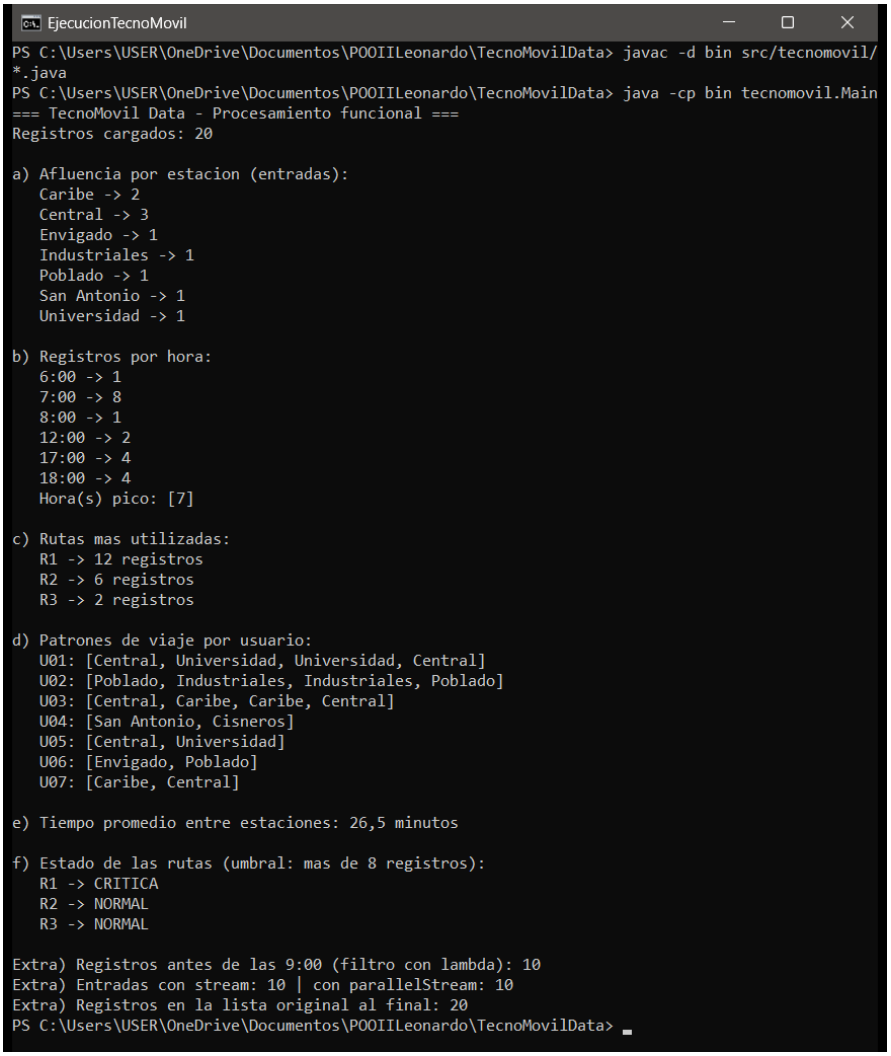
    /** Metodo de ayuda para crear registros mas corto (todos del mismo dia). */
    private static RegistroTransporte reg(String usuario, String ruta, String estacion,
                                          String accion, int hora, int minuto) {
        return new RegistroTransporte(usuario, ruta, estacion, accion,
                LocalDateTime.of(2026, 10, 30, hora, minuto));
    }
}
```

Evidencia de ejecución

El programa se compiló y ejecutó desde la terminal con los comandos javac y java. En la Figura 1 se ve la salida completa con el resultado de los seis requerimientos.

Figura 1

Compilación y ejecución del proyecto TecnoMovil Data en la terminal



Nota. Captura propia de la ejecución en Java 21.

Revisando los resultados: la estación con más entradas es Central (3); la hora pico es las 7:00 con 8 registros; la ruta más usada es R1 con 12 registros y por eso es la única marcada como

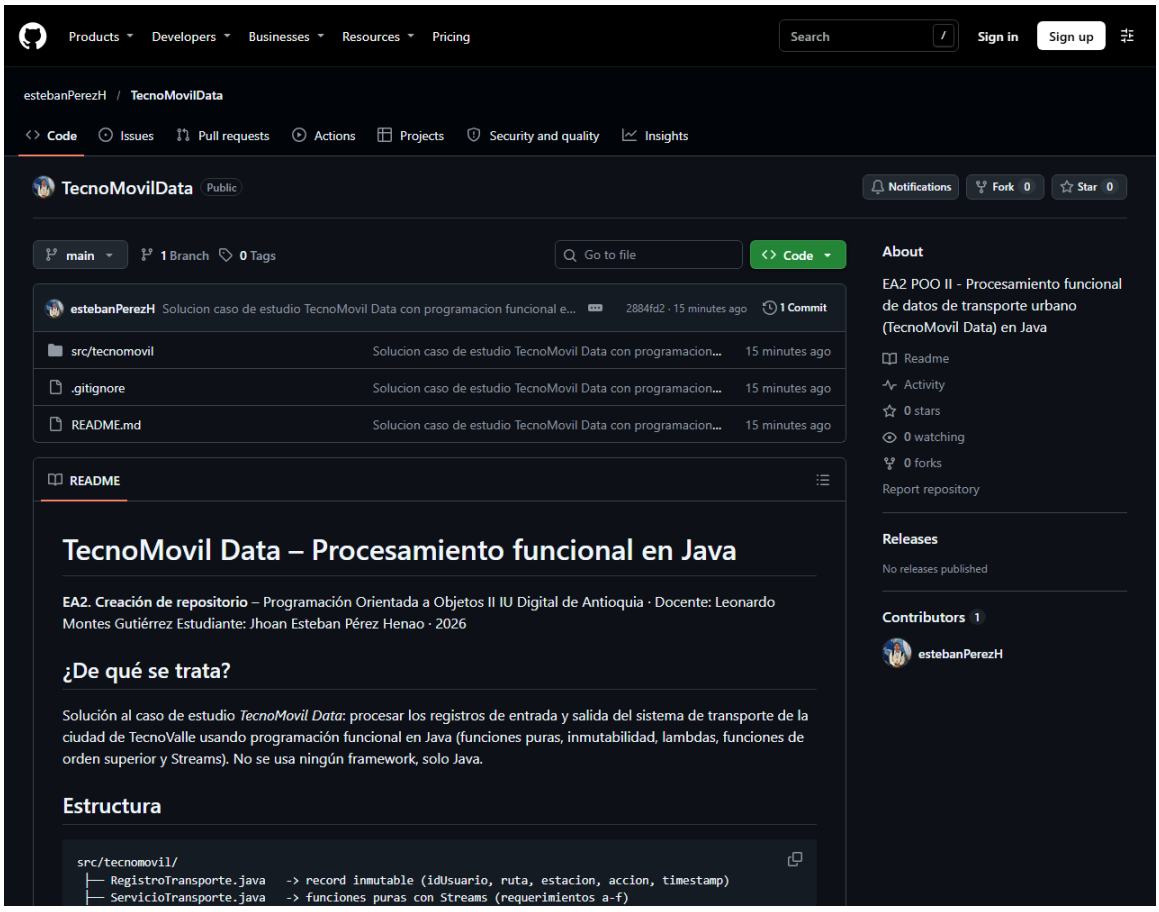
CRITICA con el umbral de 8; y el tiempo promedio de viaje entre estaciones es de 26,5 minutos. Este último lo verifiqué a mano: son 10 viajes que suman 265 minutos, y 265 / 10 = 26,5.

Repositorio en GitHub

El proyecto se subió a GitHub en la rama main. Además del código, el repositorio tiene el archivo README con las instrucciones para ejecutarlo, este informe en PDF y el archivo ZIP del proyecto final (TecnoMovilData.zip). En la Figura 2 se ve el repositorio publicado.

Figura 2

Repositorio TecnoMovilData publicado en GitHub (rama main)



Nota. Captura propia del repositorio público en GitHub.

Conclusiones

Se logró aplicar la programación funcional en un caso parecido a uno real. Los seis requerimientos de TecnoMovil Data quedaron resueltos con Streams y lambdas, y el código quedó más corto y fácil de leer que si lo hubiera hecho con ciclos y variables acumuladoras, porque cada método dice qué resultado se quiere obtener.

Entendí que la inmutabilidad y las funciones puras no son solo una forma de escribir, sino que evitan errores. Como ninguna función modifica la lista original, se pueden ejecutar todas sobre los mismos datos sin que una afecte a la otra, y por la misma razón se puede usar parallelStream sin problemas, que es justo lo que necesita un sistema que maneja millones de registros como el de TecnoValle.

Las funciones de orden superior me ayudaron a no repetir código: con contarPor resolví tres requerimientos distintos cambiando solo la lambda que le paso. Por último, la solución quedó publicada en GitHub en la rama main, lo que permite consultar el código, el informe y el ZIP desde un solo lugar.

Enlace del repositorio

Esta actividad se realizó de forma individual.

Jhoan Esteban Pérez Henao: https://github.com/estebanPerezH/TecnoMovilData

Referencias

Institución Universitaria Digital de Antioquia. (2026). Caso de estudio: procesamiento funcional de datos para un sistema de gestión de transporte urbano [Material de clase]. Programación Orientada a Objetos II.

Oracle. (2023). Package java.util.stream (Java SE 21 & JDK 21). https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/stream/package-summary.html

Oracle. (2023). Record classes. Java Language Updates. https://docs.oracle.com/en/java/javase/21/language/records.html

Urma, R.-G., Fusco, M., & Mycroft, A. (2018). Modern Java in action: Lambdas, streams, functional and reactive programming. Manning.